Stage 4: MVP Development and Execution — Sprint Plan

Project: Healthy Meals Subscription Platform (Qooti) **Duration:** June 28 – July 25, 2026 (4 Sprints × 1 Week Each)

Team Roles

Member	Administrative Role	Technical Role
Moudhi	Project Manager (PM)	Frontend Developer
Yara (Sprint 3: Reem)	Project Manager (PM)	Frontend Developer
Moudhi	Source Control Manager (SCM)	Backend Developer
Badryah	QA Lead	Database + Backend
Reem (Sprint 3: Yara)	UI/UX Coordinator	Frontend + API Integration

---

Sprint 1 — Setup and Authentication (June 28 – July 4)

Sprint Overview

In Sprint 1, the team focuses on building the foundation of the project. This includes setting up the GitHub repository with the correct branching strategy, configuring the development environment for both the frontend and backend, designing and initializing the database, and implementing the full authentication system. By the end of this sprint, any user — whether a customer, restaurant owner, or admin — should be able to register an account and log in to the platform. The Home, Register, and Login pages will be live on the frontend and connected to the backend APIs.

What We Deliver by End of Sprint

- Working GitHub repository with proper branching strategy
- Users can register and log in (customer, restaurant owner, admin)
- Database is set up and running with all core tables and seed data
- Home, Register, and Login pages are live and connected to the backend

---

Yara — PM + Frontend

Administrative tasks:

- Set up Trello board with all sprint tasks and deadlines
- Define sprint goals and assign tasks to each member
- Conduct daily stand-up meetings and document blockers
- Track sprint velocity and update task statuses daily

Technical tasks:

- Build the Register page (full name, email, password, age, gender, height, weight, health goal, delivery address)
- Build the Login page (email + password form with role-based redirect)
- Set up React project structure and routing

Moudhi — SCM + Backend

Administrative tasks:

- Create the GitHub repository
- Set up branching strategy (master, development, feature/, *merge/*)
- Enforce pull request rules: no direct push to master or development
- Review and merge the first pull requests

Technical tasks:

- Set up Python + FastAPI project
- Connect to PostgreSQL database
- Implement POST /api/auth/register (with role: customer / restaurant / admin)
- Implement POST /api/auth/login (returns JWT token + role)
- Implement POST /api/auth/logout

Badryah — QA Lead + Database

Administrative tasks:

- Write test cases for registration and login flows
- Document expected vs actual results for each test
- Set up GitHub Issues for bug tracking with severity labels (Critical / Major / Minor)

Technical tasks:

- Run schema.sql to create all database tables
- Seed mock data: 2 test restaurants, 5 test meals per restaurant, 1 admin user
- Test all auth API endpoints using Postman and document results

---

Reem — UI/UX Coordinator + Frontend + API Integration

Administrative tasks:

- Review final Figma mockups and ensure all pages match the design system
- Create a component checklist: which components are shared across pages

Technical tasks:

- Build the Home page (hero section, featured meal plans, how it works)
- Set up React Router for all main routes
- Connect Register and Login pages to POST /api/auth/register and POST /api/auth/login
- Store JWT token in app state after login
- Redirect customer to /dashboard after login
- Redirect restaurant owner to /restaurant-dashboard after login
- Redirect admin to /admin-dashboard after login

Sprint 2 — Restaurants, Meals, and Admin (July 5 – July 11)

Sprint Overview

In Sprint 2, the team builds the core features for all three user types. Restaurant owners will be able to register on the platform, log in, and fully manage their meal menu — including adding new meals, editing existing ones, and deleting unavailable ones. Customers will be able to browse the list of approved restaurants and view available meals with full nutritional details. The admin will have a dashboard to review pending restaurant registrations and either approve or reject them. All pages will be connected to their backend APIs by the end of this sprint.

What We Deliver by End of Sprint

- Restaurant owners can register, log in, add, edit, and delete meals
- Customers can browse restaurants and view available meals with full details
- Admin can view pending restaurants and approve or reject them
- All pages connected to backend APIs

Yara — PM + Frontend

Administrative tasks:

- Update Trello board with all Sprint 2 tasks
- Run daily stand-ups and track progress
- Flag any blocked tasks and help reassign if needed
- Prepare sprint review demo at end of sprint
- Review all pull requests before merging to development
- Enforce commit message standards across the team
- Resolve any merge conflicts

Technical tasks:

- Build Restaurants page (list of approved restaurants with name, description, logo)
- Add loading state on Restaurants page while fetching data
- Add empty state on Restaurants page if no restaurants are available
- Build Meal Browse page (meal cards with name, image, calories, protein, carbs, fats, price, tags)
- Add filter by restaurant on Meal Browse page
- Build Admin Overview Dashboard (stats cards: total restaurants, total customers, total orders, pending restaurants count)

Moudhi — SCM + Backend

Technical tasks:

- Implement GET /api/restaurants/:id/meals (get all meals for a restaurant)
- Implement POST /api/meals (restaurant owner adds a new meal)
- Validate all required meal fields (name, ingredients, calories, protein, carbs, fats, price)
- Implement PUT /api/meals/:id (restaurant owner updates a meal)
- Implement DELETE /api/meals/:id (soft delete: set is_available = false)
- Test POST /api/meals validates all required fields
- Test PUT /api/meals/:id updates meal correctly
- Test DELETE /api/meals/:id sets is_available to false

Badryah — QA Lead + Database

Technical tasks:

- Implement GET /api/admin/restaurants?status=pending (list pending restaurants)
- Implement GET /api/admin/restaurants (list all restaurants with status)
- Implement PATCH /api/admin/restaurants/:id/status (approve or reject + rejection reason)

- Test GET /api/restaurants returns only approved restaurants
- Test admin can approve a pending restaurant
- Test admin can reject a pending restaurant with a rejection reason

---

Reem — UI/UX Coordinator + Frontend + API Integration

Administrative tasks:

- Review UI consistency across all pages
- Ensure all pages match Figma mockups

Technical tasks:

Build restaurant management pages:

- Build Restaurant Register page (restaurant name, owner name, email, password, phone, description, logo upload)
- Build Restaurant Login page
- Build Restaurant Dashboard My Meals page (meals table with edit and delete buttons, add meal button)
- Build Add New Meal form (image upload, name, ingredients, calories, protein, carbs, fats, price, category)
- Build Edit Meal form (pre-filled with existing meal data)
- Build Admin Pending Restaurants page (table with approve and reject buttons, rejection reason modal)

Connect pages to APIs:

- Connect Restaurants page to GET /api/restaurants
- Connect Meal Browse page to GET /api/restaurants/:id/meals
- Connect Add Meal form to POST /api/meals
- Connect Edit Meal form to PUT /api/meals/:id
- Connect Delete button to DELETE /api/meals/:id
- Connect Admin Pending page to GET /api/admin/restaurants?status=pending
- Connect Approve button to PATCH /api/admin/restaurants/:id/status
- Connect Reject button to PATCH /api/admin/restaurants/:id/status with rejection reason

---

Sprint 3 — Subscription, Orders, and Payment (July 12 – July 18)

Sprint Overview

In Sprint 3, the team builds the complete customer journey from meal selection to payment. Customers will be able to select one meal per day for the subscription week (Sunday to Thursday), review their full order, enter their delivery time, apply a discount code, and complete payment. Restaurant owners will be able to view their incoming orders grouped by day of the week so they can prepare meals in advance. By the end of this sprint, the full end-to-end flow — from registration to placing a paid order — will be functional.

What We Deliver by End of Sprint

- Customers can select meals, review their order, apply a discount code, and complete payment
- Restaurant owners can view incoming orders grouped by day
- Full end-to-end customer journey is functional and tested

Reem — PM + Frontend

(Role swap: Reem takes PM this sprint)

Administrative tasks:

- Monitor sprint velocity and identify tasks at risk
- Adjust task assignments if any member is blocked
- Document daily stand-up notes and blockers

Technical tasks:

- Build Weekly Meal Selection page (Sunday to Thursday tabs, one meal per day, validation before proceeding)
- Build Order Summary page (selected meals per day, delivery time input, discount code input with Apply button, price breakdown)
- Build Customer Dashboard page (active subscription details, weekly schedule, subscription status)

Moudhi — SCM + Backend

Administrative tasks:

- Review all pull requests
- Ensure development branch is stable before each merge
- Prepare development branch for Sprint 4 final integration

Technical tasks:

- Implement POST /api/subscriptions (create subscription with start_date, end_date, delivery_time, discount_code_id)
- Implement POST /api/meal-selections (save selected meal per day: subscription_id, meal_id, day_date, day_of_week)
- Reject duplicate day selection in the same subscription
- Implement POST /api/payments (process payment and update subscription status to confirmed)
- Create payment row with status = pending on subscription creation
- Update payment status to success or failed based on payment response

---

Badryah — QA Lead + Database

Technical tasks:

- Implement GET /api/restaurants/:id/orders?day=Sunday (restaurant orders grouped by day)
- Validate subscription date ranges before inserting order items
- Test POST /api/subscriptions creates subscription correctly
- Test POST /api/meal-selections saves one meal per day
- Test duplicate day selection is rejected
- Test discount code reduces final price correctly
- Test POST /api/payments confirms subscription status
- Test full subscription and payment in one database transaction

---

Yara — UI/UX + Frontend + API Integration

(Role swap: Yara takes frontend this sprint)

Administrative tasks:

- Final UI review for all Sprint 3 pages
- Ensure all new pages are responsive

Technical tasks:

- Build Payment page (order summary on left, payment form on right: cardholder name, card number, expiry, CVV)
- Add Pay Now button and success confirmation message on Payment page

- Build Restaurant Orders by Day page (day selector tabs Sunday to Thursday, orders table per day)
- Connect Meal Selection page to GET /api/restaurants/:id/meals
- Connect Order Summary to POST /api/subscriptions and POST /api/meal-selections
- Connect discount code Apply button to POST /api/discount-codes/validate
- Connect Payment page to POST /api/payments
- Connect Restaurant Orders page to GET /api/restaurants/:id/orders

---

Sprint 4 — Testing, Bug Fixes, and Final Delivery (July 19 – July 25)

Sprint Overview

Sprint 4 is the final sprint. The team shifts focus from building new features to testing, fixing bugs, and preparing the project for final submission. Every user flow will be tested end-to-end across all three user types. All critical bugs will be fixed. The backend will be deployed to a production server and the frontend will be deployed and accessible via a public URL. The team will also prepare all required submission documents including the final README, bug report, testing evidence, and sprint retrospective.

What We Deliver by End of Sprint

- Fully tested and stable MVP deployed to production
- Clean GitHub repository with organized code and documentation
- Final README, bug report, and testing evidence ready for submission

---

Member 1 — PM + Frontend

Administrative tasks:

- Conduct final sprint review and demo all features to the team
- Run sprint retrospective (what went well, what did not, improvements)
- Collect all submission links (repo, Trello, bug tracker, production URL, testing evidence)
- Prepare final deliverables document for submission

Technical tasks:

- Fix all frontend bugs reported during testing
- Review and fix spacing, colors, and typography on all pages
- Ensure all pages are responsive on different screen sizes
- Test all navigation links and buttons work correctly
- Take screenshots of all pages for documentation

Member 2 — SCM + Backend

Administrative tasks:

- Final code review of all pull requests
- Merge development branch into master after all tests pass
- Tag the final release on GitHub (v1.0.0)

Technical tasks:

- Fix all backend bugs reported during testing
- Optimize slow database queries
- Set up environment variables for production
- Deploy backend to Render or Railway
- Deploy frontend to Vercel or Netlify
- Verify production deployment is working correctly

Member 3 — QA Lead + Database

Administrative tasks:

- Write final bug report (all bugs found, severity, status: fixed or open)
- Export Postman collection as testing evidence
- Take screenshots of all passing tests
- Conduct User Acceptance Testing for all three user types

Technical tasks:

- Run end-to-end tests for customer flow (register → login → browse → select meals → pay)
- Run end-to-end tests for restaurant flow (register → login → add meals → view orders)
- Run end-to-end tests for admin flow (login → approve or reject restaurant)
- Test production environment after deployment
- Seed production database with realistic mock data

Member 4 — UI/UX + Frontend + API Integration

Administrative tasks:

- Final UI review against original Figma mockups

- Document any design decisions that changed during development

Technical tasks:

- Connect any remaining pages to their APIs
- Fix all API integration bugs
- Add error messages on all forms
- Add loading spinner while waiting for API responses
- Handle empty states on all pages (no restaurants, no meals, no orders)

Sprint Summary

Sprint	Dates	Key Goals	Milestone
Sprint 1	Jun 28 – Jul 4	Project setup · Auth APIs · Register + Login pages · Database	Users can register and log in
Sprint 2	Jul 5 – Jul 11	Restaurant registration · Meal management · Customer browsing · Admin approval	Restaurants can manage meals, admin can approve
Sprint 3	Jul 12 – Jul 18	Meal selection · Order summary · Payment · Restaurant orders view	Full customer journey complete
Sprint 4	Jul 19 – Jul 25	End-to-end testing · Bug fixes · Deployment · Final submission	MVP live on production

---

Definition of Done

A task is considered done when:

- Code is written and tested locally
- A pull request is opened and reviewed by at least one team member
- QA has tested the feature and confirmed no critical bugs
- The feature is merged to the development branch